Code Coverage Report

Blood Bank Management System (BBMS) v1.0 - Backend

Project	SE Mini Project - Team 9, Blood Bank Management System
Repository	blood-bank-management-team-9/se-mini-project-team9-blood-bank
Branch tested	feature/18-record-donation (last confirmed remote commit 60a5136)
Related issue	Upload Code Coverage Report (issue number to be confirmed)
Governing document	BBMS Software Test Plan v1.0 (03-10-2026), Test Execution Cycle 2: 02-11-2026 to 06-11-2026
Coverage run date	5 November 2026 (Istanbul report generated 2026-11-05T17:39:35Z)
Prepared by	Test Engineer (Chukkapalli Rohan)
Status	Draft for QA Lead review

97.91%	90.69%	98.63%	99.05%
Statements	**Branches**	**Functions**	**Lines**
1408 of 1438	780 of 860	217 of 220	1357 of 1370

All four metrics exceed the 70% unit-test coverage criterion in the STP (NFR-QUAL-04, TC-37 step 2).

1. Introduction

This report presents the code coverage achieved by the automated backend unit test suite of the Blood Bank Management System (BBMS). It is one of four test reports required by the project's testing assignment (Unit Test Report, System Validation Report, Code Coverage Report and Branch Coverage Report) and is prepared under the BBMS Software Test Plan v1.0 (STP), which follows IEEE Std 829-1998.

Scope. The report covers the Node.js backend source under backend/src, as measured by the Jest coverage run of 5 November 2026. It does not cover the front end, performance, security, usability or system-level behaviour; those are addressed by other test types in the STP.

What the four metrics measure. Jest reports coverage using Istanbul, which measures four metrics:

- **Statements** - the percentage of executable statements run at least once by the tests.
- **Branches** - the percentage of decision outcomes (if/else, switch cases, ternaries, short-circuit expressions, default values) exercised. This is analysed in detail in the separate Branch Coverage Report.
- **Functions** - the percentage of declared functions called at least once.
- **Lines** - the percentage of source lines containing executable code that were run at least once.

Coverage shows which code was executed by the tests. It does not show that the assertions are sufficient or that the code is correct.

2. Alignment with the Software Test Plan

The only coverage criterion the STP defines is in the quality requirement and test case below.

STP item	Requirement	Relevance to this report
NFR-QUAL-04 (Maintainability and testability)	Code organised in per-feature modules; unit tests cover at least 70% of business logic.	Measured coverage is well above 70% on every metric. Per-feature modules are visible in the module breakdown (Section 6).
TC-37, step 2	Run the unit test suite with a coverage report. Expected: unit tests cover at least 70% of business logic.	This report and the supporting HTML coverage report are the evidence for step 2.
TC-37, step 1 (NFR-QUAL-02)	Monitor uptime over a calendar month; availability at least 99%.	Not covered here. This needs separate evidence, so TC-37 should not be marked fully passed on the basis of this report alone.
STP Section 5, Unit tests	Business logic such as eligibility calculation, expiry date calculation and stock decrement, using Jest.	Matches the tooling and scope of the suite measured here.
STP Section 5, Exit criteria	100% of TC-01 to TC-37 executed; 0 critical or high defects open; RTM complete.	Coverage is one input to these criteria. It does not show that all 37 test cases were executed.

The STP's 70% criterion does not name a metric, so this report applies it to statements, branches, functions and lines. All four exceed it. The STP describes the threshold as applying to business logic; the figures reported here are for the whole backend source measured by the Jest run, which also includes infrastructure code such as the database layer.

3. Methodology and Tools

- **Test framework:** Jest, as specified in STP Section 6 (Tools).
- **Coverage instrumentation:** Jest's built-in coverage collection (Istanbul) with the standard HTML (lcov-report) reporter.
- **Command used:** npm.cmd test -- --coverage, run from the backend directory on Windows PowerShell.
- **Test style:** Unit tests running against mocks or isolated test setups (see Section 8).

- **Report conversion:** The Istanbul HTML report was saved to PDF from the browser for submission; a screenshot of the original report is reproduced in Appendix A.

4. Test Environment

Item	Detail
Machine	Developer laptop, Windows, PowerShell (local repository at the team member's workstation)
Source branch	feature/18-record-donation
Coverage report generated	2026-11-05T17:39:35Z by Istanbul (Jest HTML lcov-report)
Browsers / system tests	Not applicable to this report.

5. Scope of the Measured Run

The figures in this report come from the Jest coverage run whose Istanbul report was generated on 5 November 2026 at 17:39 UTC. The measured source includes the config, database, middleware, admin, audit, auth, donations, donors, health, reports, search and stock modules. Pass/fail/skip counts for the individual tests are recorded in the Unit Test Report and are not repeated here.

6. Coverage Results

6.1 Overall coverage

Metric	Covered / Total	Uncovered	Percentage	STP threshold (70%)
Statements	1408 / 1438	30	97.91%	Met
Branches	780 / 860	80	90.69%	Met
Functions	217 / 220	3	98.63%	Met
Lines	1357 / 1370	13	99.05%	Met

6.2 Module-wise coverage

Module	Statements	Branches	Functions	Lines
src	95.55% (43/45)	78.57% (11/14)	100% (1/1)	97.72% (43/44)
src/config	100% (5/5)	100% (0/0)	100% (0/0)	100% (5/5)
src/database	91.3% (21/23)	94.44% (17/18)	33.33% (1/3)	100% (21/21)
src/middleware	95.18% (79/83)	92.85% (52/56)	93.33% (14/15)	97.46% (77/79)
src/modules/admin	100% (141/141)	100% (71/71)	100% (23/23)	100% (140/140)
src/modules/audit	100% (12/12)	100% (13/13)	100% (2/2)	100% (12/12)
src/modules/auth	100% (155/155)	100% (75/75)	100% (30/30)	100% (147/147)
src/modules/donations	99.09% (218/220)	94.07% (143/152)	100% (38/38)	99.5% (203/204)

Module	Statements	Branches	Functions	Lines
src/modules/donors	97.92% (189/193)	96.07% (98/102)	100% (35/35)	98.38% (183/186)
src/modules/health	100% (13/13)	100% (4/4)	100% (1/1)	100% (13/13)
src/modules/reports	95.44% (314/329)	79.84% (202/253)	100% (29/29)	98.05% (303/309)
src/modules/search	99.01% (101/102)	88.52% (54/61)	100% (11/11)	100% (98/98)
src/modules/stock	100% (117/117)	97.56% (40/41)	100% (32/32)	100% (112/112)
Total	**97.91%** (1408/1438)	**90.69%** (780/860)	**98.63%** (217/220)	**99.05%** (1357/1370)

Per-module counts sum to the overall totals. Cell shading: green 80% or above, yellow 50-79%, red below 50%. Modules with a 0/0 entry (config for branches and functions, health for none) have no such items, so their 100% reflects an empty denominator.

6.3 Distribution of uncovered items

Module	Statements	Branches	Functions	Lines
src/modules/reports	15 (50%)	51 (64%)	0	6 (46%)
src/modules/donations	2 (7%)	9 (11%)	0	1 (8%)
src/modules/search	1 (3%)	7 (9%)	0	0
src/middleware	4 (13%)	4 (5%)	1 (33%)	2 (15%)
src/modules/donors	4 (13%)	4 (5%)	0	3 (23%)
src	2 (7%)	3 (4%)	0	1 (8%)
src/database	2 (7%)	1 (1%)	2 (67%)	0
src/modules/stock	0	1 (1%)	0	0
Total uncovered	**30**	**80**	**3**	**13**

The reports module accounts for half of the uncovered statements and almost two thirds of the uncovered branches, so it is the most efficient place to add tests. Modules not listed (config, admin, audit, auth, health) have no uncovered items.

7. Uncovered Code Analysis

Module-level counts come directly from the Istanbul report. Line-level detail is available in the per-file pages of the HTML report (docs/jest-coverage-html/). Uncovered code reflects tests that were not written; it is not evidence of defects in the application.

Area	Observation
src/modules/reports (95.44% statements, 79.84% branches)	Largest gap: 15 uncovered statements, 51 uncovered branches and 6 uncovered lines. All 29 functions are covered, so the gap is in alternative paths within functions (conditions, filters and error paths) rather than untested functions.
src/modules/search (88.52% branches)	7 of 61 branches not exercised; statements and lines are essentially fully covered.
src (78.57% branches)	3 of 14 branches uncovered in application bootstrap code; a small sample, so the percentage swings widely.

Area	Observation
src/database (33.33% functions)	1 of 3 functions covered. Statement (91.3%) and line (100%) coverage are high, so the uncovered functions are likely not exercised directly by the tests; confirm in the HTML report.
src/middleware (93.33% functions)	1 uncovered function, 4 uncovered statements and 4 uncovered branches.
src/modules/donations and donors	9 and 4 uncovered branches respectively, with a few uncovered statements and lines in each.

8. Limitations

- **Mocked behaviour.** Passing unit tests with mocks show that the logic behaves correctly given the assumed responses of its dependencies. They do not show that the real database, SQL or transaction handling behaves the same way.
- **No integration or system evidence.** Atomic-transaction behaviour (TC-33), backup and restore (TC-34), performance (TC-17), TLS (TC-35) and browser compatibility (TC-36) are not demonstrated by this report. Do not read 97% coverage as complete database integration or end-to-end validation.
- **Coverage is not correctness.** Coverage shows that code was executed, not that the assertions are sufficient.
- **Whole-backend figures.** The figures cover all backend source, not only business logic as worded in the STP criterion.
- **Mapping to test cases.** The suite is organised by module, not by STP test case ID (TC-01 to TC-37), so this report does not claim any specific STP test case as passed.

9. Conclusion

The backend unit test suite achieves 97.91% statement, 90.69% branch, 98.63% function and 99.05% line coverage. All four values exceed the 70% unit-test coverage criterion in the STP (NFR-QUAL-04, TC-37 step 2). The admin, audit, auth, config and health modules are fully covered on statements, functions and lines.

Branch coverage is the lowest metric, driven mainly by the reports module (79.84%) and, to a lesser extent, search (88.52%). The only function-level gap is in the database layer (1 of 3 functions covered) and one middleware function.

Recommended follow-up (optional, before submission):

- Add tests for the uncovered branches in the reports and search modules.
- Add tests that exercise the remaining database and middleware functions.
- Confirm line-level details against the HTML report.

10. Supporting Evidence and Artifacts

Artifact	Location
HTML coverage report (entry page)	docs/jest-coverage-html/index.html
Original generated report	backend/coverage/lcov-report/index.html
This report	docs/Jest_Coverage_Report.pdf
Software Test Plan	docs/BBMS_Software_Test_Plan_v1.0.pdf
Command	npm.cmd test -- --coverage

11. Review and Sign-off

Role	Name	Signature / Date
Test Engineer	Chukkapalli Rohan	
QA Lead	Dheepak Seetharam	
Product Owner	Harsimran Kaur	

Appendix A - Istanbul HTML Report (Screenshot of backend/coverage/lcov-report/index.html)

All files

97.91% Statements `1408/1438` **90.69%** Branches `780/860` **98.63%** Functions `217/220` **99.05%** Lines `1357/1370`

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	95.55%	43/45	78.57%	11/14	100%	1/1	97.72%	43/44
src/config	100%	5/5	100%	0/0	100%	0/0	100%	5/5
src/database	91.3%	21/23	94.44%	17/18	33.33%	1/3	100%	21/21
src/middleware	95.18%	79/83	92.85%	52/56	93.33%	14/15	97.46%	77/79
src/modules/admin	100%	141/141	100%	71/71	100%	23/23	100%	140/140
src/modules/audit	100%	12/12	100%	13/13	100%	2/2	100%	12/12
src/modules/auth	100%	155/155	100%	75/75	100%	30/30	100%	147/147
src/modules/donations	99.09%	218/220	94.07%	143/152	100%	38/38	99.5%	203/204
src/modules/donors	97.92%	189/193	96.07%	98/102	100%	35/35	98.38%	183/186
src/modules/health	100%	13/13	100%	4/4	100%	1/1	100%	13/13
src/modules/reports	95.44%	314/329	79.84%	202/253	100%	29/29	98.05%	303/309
src/modules/search	99.01%	101/102	88.52%	54/61	100%	11/11	100%	98/98
src/modules/stock	100%	117/117	97.56%	40/41	100%	32/32	100%	112/112

Code coverage generated by istanbul at 2026-11-05T17:39:35.574Z